

# **Getting started with Human-in-the-loop Machine Learning**

Dr. Mohammed Brahimi and Dr. Aicha Boutorh

# Agenda

- Ranking predictions by model confidence to identify confusing items
- Finding unlabeled items with novel information
- Building a simple interface to annotate training data
- Evaluating changes in model accuracy as you add more training data

# Introduction

For any machine learning task, you should start with a **simple** but **functional** system and build out more sophisticated components as you go.

This guideline applies to most technology: **ship the minimum viable product (MVP)** and then iterate on that product.

The **feedback** you get from what you ship first will tell you which pieces are the most important to build out next.

# First active learning algorithm

- Active learning is the process of selecting the **right** data for human review.
- You've likely used basic forms of active learning (e.g., *keyword filtering*).

## The Hidden Filtering in Common Datasets

- Popular academic datasets (*ImageNet*, *MNIST*, *CoNLL*) are often **heavily pre-filtered**.
- **Random sampling** from these datasets isn't truly random; it reflects the original (potentially outdated) sampling strategies.
- You might be unknowingly using **heuristics** from a decade ago.

# Data Challenges & Strategic Learning in ML

## The Challenge of Dataset Provenance

- **Poor Documentation:** Formal records on how common datasets were created or filtered are often missing.
- **Opaque Processes:** Understanding their origin is difficult and often arbitrary.

***Don't assume existing datasets are perfectly representative of real-world data.***

## ML as an Ongoing, Active Learning Process

- **Early Decisions Matter:** Data sampling choices are made *before* model building—even if suboptimal.
- **Strategic Sampling Forward:** Treat model development as a **continuous process**—actively refine your dataset.

***Continuously evaluate and improve data collection to boost model performance.***

# From Ad Hoc to Algorithm: Your First System

## Hacktive Learning

- Using informal, quick fixes to sample data?  $\Rightarrow$  “**hacktive learning**”
- **Tempting but risky**: Shortcuts can mislead your model and your process.

## Move Toward Fundamentals

- Build understanding, not just quick wins.
- Focus on **principled data sampling** and system design.

## Your First System Architecture

- Will likely involve a **human-in-the-loop** component.
- Follows a **basic feedback-driven architecture** (see Figure 2.1).

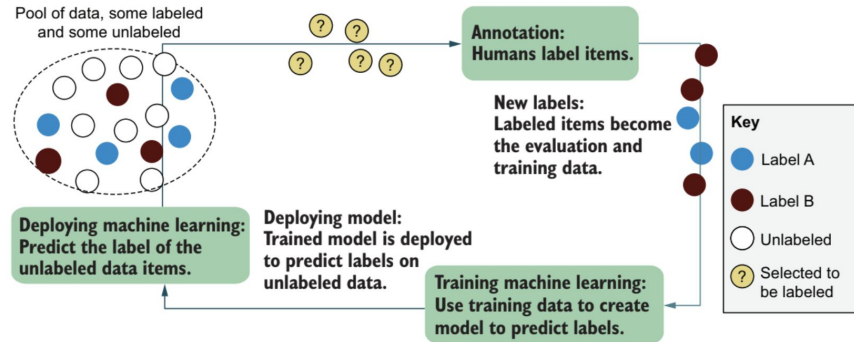


Figure 2.1 The architecture of your first human-in-the-loop machine learning system

# First example: Classifying Disaster-Related News Headlines

**Task:** Build a **human-in-the-loop** system to label news headlines as:

-  *Disaster-related*
-  *Not disaster-related*

## Key Use Cases

- **Real-Time Crisis Response**
  - Identify disaster-related news to assist emergency response teams
- **Enhanced News Searchability**
  - Tag articles for better indexing in media databases
- **Media & Social Research**
  - Enable studies on disaster coverage in journalism



# A Simple Task with Life-Saving Potential

## Real-World Example

- Early detection of disease outbreaks (e.g., **H5N1**, **H1N1**)
- Timely alerts help **epidemiologists respond faster**
- Even simple systems can have **critical impact**

## Beyond Binary Classification

- Real disasters require **fine-grained labels** (e.g., *food*, *water*, *shelter*)
- **Prioritization** matters (e.g., *urgent water needs*)
- Different organizations address **specific needs**

## Why Start Simple?

- “Relevant” vs. “Not relevant” is a **crucial first filter**
- Helps route data to the right people and tools



## Expert Insight: Peter Skomoroch

*“You need to look at real data in depth to know exactly what models to build.”*

### Go Beyond the Dashboard

- High-level charts and stats are **not enough**
- Regularly **examine granular, randomly sampled data**
- Let real examples “**wash over you**” to build intuition

### Build Data Instincts

- Just like:
  - Executives review **weekly KPIs**
  - Engineers monitor **system logs**
- Data scientists should **track how their data evolves over time**

***Deep familiarity with raw data is critical for building meaningful and effective ML systems.***



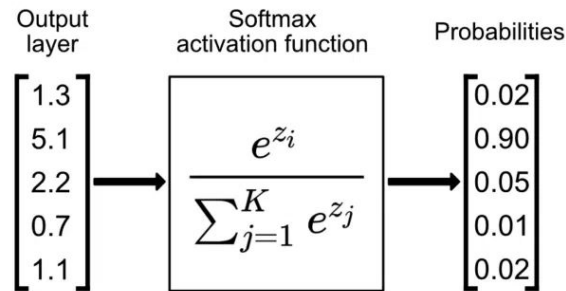
# Interpreting model predictions and data to support active learning

## Key Outputs of Supervised Machine Learning Models:

1. **Predicted Label** (or Set of Predictions)
2. **Confidence Scores** (or Set of Numbers) Associated with Each Prediction

## Understanding Uncertainty in Predictions:

- **Uncertainty Signals Value:**
  - When a model shows similar confidence scores for mutually exclusive categories, it indicates **model confusion**.
- **Human Judgment Matters:**
  - Human input is **most valuable** when the model is uncertain, providing insight into these ambiguous cases.
- **Training Benefits:**
  - **Training the model** on these uncertain examples results in the **greatest learning gains** for the model's performance.



# Evaluating Label Accuracy in Traditional Machine Learning

## Disaster-Related Message Prediction

- **Prediction Outcome:**
  - The model predicts: **"Not Disaster-Related"**

## Traditional ML Evaluation Focus:

- **Label Accuracy on Held-Out Data:**
  - Traditional machine learning focuses on determining whether the predicted label is **correct** based on the actual label.

## Key Questions in Traditional ML:

1. Was the label prediction correct?
2. *What is the overall accuracy of the model when predicting on a large held-out dataset?*

```
{  
  "Object": {  
    "Label": "Not Disaster-Related",  
    "Scores": {  
      "Disaster-Related": 0.475524352,  
      "Not Disaster-Related": 0.524475648  
    }  
  }  
}
```

# Active Learning Focus: Confidence Scores

## Confidence Over Prediction:

- In active learning, **confidence scores** matter more than the predicted label itself.
- Example: The model predicts **"Not Disaster-Related"** with a **52.4% confidence**—indicating **uncertainty**.

## Value of Human Review:

- Despite the "Not Disaster-Related" prediction, the **47.6% confidence** for "Disaster-Related" suggests the model may be unsure.
- **Human review** is needed to verify the prediction and correct errors.
- **Adding uncertain examples** to the training data helps the model **learn** and **improve**.

```
{  
  "Object": {  
    "Label": "Not Disaster-Related",  
    "Scores": {  
      "Disaster-Related": 0.475524352,  
      "Not Disaster-Related": 0.524475648  
    }  
  }  
}
```

# Active Learning: Confidence Scores

## New Prediction Example:

- The model predicts **"Not Disaster-Related"** with **98.4% confidence**.
- **Comparing Confidence Levels:**
  - **52.4%** confidence for "Not Disaster-Related"—**indicating uncertainty**.
- Second Example:
  - **98.4%** confidence—indicating **high certainty**.

*The model is much more confident in the second prediction, suggesting higher reliability for this prediction.*

```
{
  "Object": {
    "Label": "Not Disaster-Related",
    "Scores": {
      "Disaster-Related": 0.015524352,
      "Not Disaster-Related": 0.984475648
    }
  }
}
```

# Focusing Review on Uncertain Model Predictions

## Prioritizing Low-Confidence Items for Review

- **Low Confidence = Higher Risk of Mislabeling:**
  - Items with **lower confidence** (e.g., 52.4%) are more likely to be mislabeled.
  - The first item should be **prioritized** for **human review**.
- **Improving Model Learning:**
  - Addressing **uncertain cases** helps the model **learn more effectively**.

## The Reliability of Rank Order

- **Confidence Rank Order:**
  - Even if confidence percentages aren't perfectly accurate, the **rank order** is usually reliable.
  - **Lower confidence** items are generally more likely to be **incorrect**.
- **General Rule:**
  - This holds for most **machine learning algorithms** and **accuracy metrics**.

# Confidence Calculation for Most Confident Label

For a set of labels ( $\mathbf{y}$ ) and an item ( $\mathbf{x}$ ), the confidence  $\phi\mathbf{c}(\mathbf{x})$  is the probability of the most confident label ( $\mathbf{y}^*$ ):

$$\phi\mathbf{c}(\mathbf{x}) = P_{\theta}(\mathbf{y}^* \mid \mathbf{x})$$

Where  $\theta$  represents the model parameters.

## Simple Strategy for Binary Tasks

- For **binary classification** (like disaster relevance task), simply rank items by their **confidence score**.
- **Sample** the items with **confidence closest to 50%**. These are the most **uncertain** predictions.

# Confidence in Complex vs. Simple Tasks

For tasks with more than two labels, sequence labeling, generation tasks, or object detection, confidence calculation becomes **more complex**.

- **Core Intuition Remains**

Low confidence = High uncertainty → Needs **human input**.

- **Start Simple: Binary Tasks**

- Easier confidence estimation
- Ideal for initial **human-in-the-loop** system
- Focus on using **confidence ranking** for **active learning**

# Identifying Outliers: Ensuring Diversity (here)

- **Goal:** Select a **diverse** set of items for human labeling
  - Avoid sampling only similar examples
  - Capture **rare but important** edge cases
- **Why It Matters?**
  - Diversity helps models generalize and recognize less frequent patterns
- **Example – The Asteroid Outlier**  
**Headline:** *“Asteroid flattens Walnut Creek”*
  - May confuse the model:
    - **Unfamiliar term:** “asteroid”
    - **Uncommon location:** “Walnut Creek”
  - Represents a **true outlier** — far from training data

# Simple Outlier Detection: Word Frequency

## Algorithm: Outlier Sampling by Word Frequency

1. **Count:** For each **unlabeled item**, compute the **average number of word matches** with items in the training set.
2. **Rank:** Sort items by match count — **lowest first** (i.e., least similar to known data).
3. **Sample:** Pick the item with the **lowest average match** — likely an **outlier**.
4. **Label:** Mark the sampled item for **human annotation**.
5. **Repeat:** Continue until you've selected enough items for a **review iteration**.

# Simple Outlier Detection: Word Frequency

## 1. Favors short, novel headlines

- A 3-word sentence with 0 matches looks “more novel” than a 10-word sentence with 2 rare matches and 8 common ones.
- The method prioritizes **fewer total matches**, so **shorter items** are more likely to appear diverse.

## 2. Smoothing (+1) is used

- Typically used to avoid rank tie issues.
- But it may **reduce the contrast** between really novel and slightly novel examples.

## 3. Disfavors examples with many common words

- Even unlabeled sentence with **some very rare or informative words**, the overall match count may still be high due common words.
- So, useful data may be **missed** because it **shares too much surface form** with prior examples.

# More Sophisticated Outlier Detection

## Refinement 1: Bayesian Outlier Scoring

$$P(\text{sentence}) = \prod_{w \in \text{sentence}} P(w)$$

with:

$$P(w) = \frac{\text{count}(w) + 1}{\text{total words} + |V|}$$

Low probability  $\Rightarrow$  high novelty

## Refinement 2: Edit Distance & Similarity Metrics

- Measures **structural novelty**, e.g.:
  - Levenshtein distance, Cosine similarity (TF-IDF)
  - Jaccard similarity (word sets)

*Start simple (word match), then experiment with probabilistic and distance-based methods to better surface **truly novel, informative examples.***

# Example: Bayesian Outlier Scoring

## Labeled Sentences:

1. "the cat sleeps"
2. "a dog runs"
3. "the mouse eats"

## Unlabeled Sentence A: "a lion sleeps quietly"

Word Frequencies (with +1 smoothing, total words = 9, vocab = 8):  $P(w) = \frac{\text{count}(w) + 1}{17}$

A  $\Rightarrow 1 \Rightarrow 2/17 \approx 0.118$

lion  $\Rightarrow 0 \Rightarrow 1/17 \approx 0.059$

sleeps  $\Rightarrow 1 \Rightarrow 2/17 \approx 0.118$

quietly  $\Rightarrow 0 \Rightarrow 1/17 \approx 0.059$

$$P = 0.118 * 0.059 * 0.118 * 0.059 \approx 0.000048$$

# What is Levenshtein Distance?

The **minimum number of single-character edits** (insertions, deletions, substitutions) needed to change one string into the other.

- **Original Sentence 1 :** cat sleeps
- **Substitute 't' → 'ts' :** cats sleeps
- **Delete 's' from 'sleeps':** cats sleep

Total operations = 2

The **Levenshtein distance** between "cat sleeps" and "cats sleep" is 2.

*This means they are quite similar, with just two small edits needed*

## OPERATIONS

1. REPLACE
2. INSERT
3. DELETE

## OPERATIONS

|         |        |
|---------|--------|
| replace | delete |
| insert  | (i, j) |

CURRENT POSITION

|     |   | a | r | e | p | e | a | c | e |
|-----|---|---|---|---|---|---|---|---|---|
| " " | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |   |
| 1 d | 1 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |   |
| 2 e | 2 | 2 | 1 | 2 | 3 | 4 | 5 | 6 |   |
| 3 l | 3 | 3 | 2 | 2 |   |   |   |   |   |
| 4 e | 4 |   |   |   |   |   |   |   |   |
| 5 t | 5 |   |   |   |   |   |   |   |   |
| 6 e | 6 |   |   |   |   |   |   |   |   |

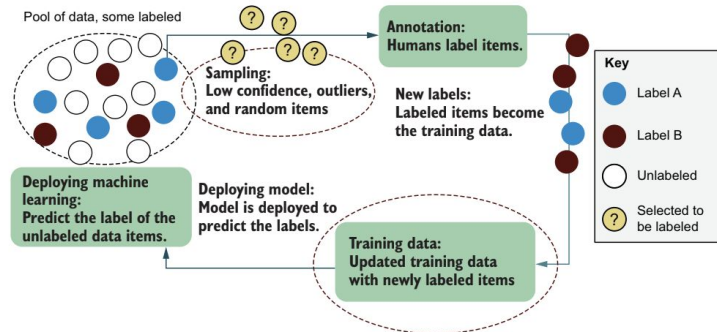
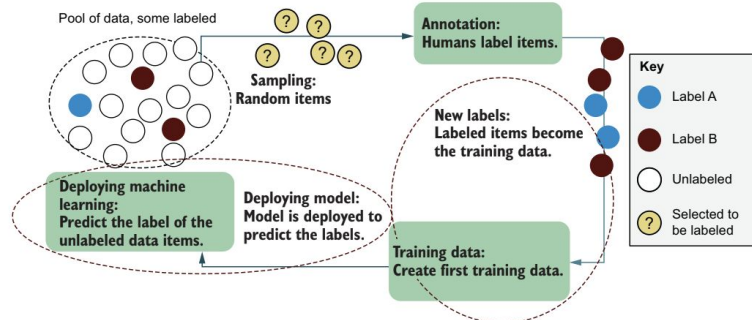
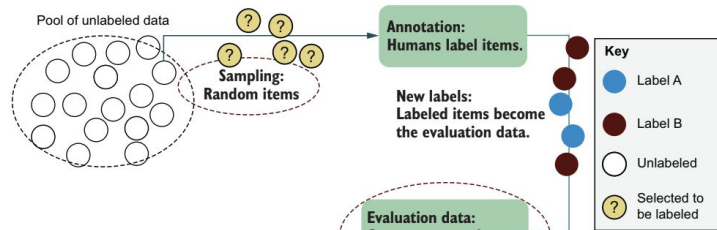
LEETCODE

# EDIT DISTANCE

THE LEVENSHTein DISTANCE

# Building Your First Human-in-the-Loop Mod

1. Top: annotate a **random sample** of to set aside as your **evaluation data**.
2. Label the first items to be used for **training data** (middle), also starting with a **random selection**.
3. Start using **active learning** (bottom) to sample items that are **low-confidence** or **outliers**.



# Iterating with Active Learning — What to Expect

- Example code iterates every 100 items.
- This number is **small for efficiency** but **good for observing changes**.
- Expect to see patterns and improvements over iterations.

## First Iteration — Initial Annotation

- Labeling mostly "**not disaster-related**" headlines
- Build a **randomly sampled evaluation set**
- Challenges:
  - Metaphors (e.g., "hurricane of goals")
  - Edge cases (e.g., is a plane crash a disaster?)
- Helps refine **task definition** and instructions

# Iterating with Active Learning — What to Expect

## Second Iteration — First Model

- First model is trained
- Low F-score ( $\sim 0.20$ ), better AUC ( $\sim 0.75$ )
- Model leans toward **majority class**, but...
- Active learning helps **self-correct**
  - Surfaces uncertain, minority-class (disaster) items

**Risk: Overfitting on early random examples (e.g., many flood-related headlines)**

## 3rd & 4th Iterations — Accuracy Improves

- More **"disaster-related"** items are labeled
- Label balance trends toward **50:50**
- Model starts learning counterexamples
  - e.g., "investment floods the market"  $\neq$  disaster
- Diversity of samples increases

# Iterating with Active Learning — What to Expect

## 5th–10th Iterations — Solid Progress

- Accuracy continues to improve
- F-score/AUC grow **by a few points per 100 labels**
- More diverse and meaningful headline sampling
- **Limitation: Lack of additional random evaluation data**

***Hard to go back to random sampling without losing prior labels***

# Building an interface to get human labels

## Boosting Efficiency

- A **faster, smoother interface** can improve results as much as better algorithms.
- **50% better interface  $\approx$  50% better sampling strategy**

## Prioritize the Human Annotator

- **Respect** the time and effort of the people labeling your data.
- Strive to make them feel as **effective** as possible.
- **Prioritize interface improvements** over algorithmic ones

## Real-World Complexity

- Add **quality control** to catch annotation mistakes
- Refine labels and instructions iteratively
- Support **multiple concurrent annotators** with good task management systems

# Building an interface to get human labels

## Boosting Efficiency

- A **faster, smoother interface** can improve results as much as better algorithms.
- **50% better interface  $\approx$  50% better sampling strategy**

## Prioritize the Human Annotator

- **Respect** the time and effort of the people labeling your data.
- Strive to make them feel as **effective** as possible.
- **Prioritize interface improvements** over algorithmic ones

## Real-World Complexity

- Add **quality control** to catch annotation mistakes
- Refine labels and instructions iteratively
- Support **multiple concurrent annotators** with good task management systems

# Building an interface to get human labels

## Boosting Efficiency

- A **faster, smoother interface** can improve results as much as better algorithms.
- **50% better interface  $\approx$  50% better sampling strategy**

## Prioritize the Human Annotator

- **Respect** the time and effort of the people labeling your data.
- Strive to make them feel as **effective** as possible.
- **Prioritize interface improvements** over algorithmic ones

## Real-World Complexity

- Add **quality control** to catch annotation mistakes
- Refine labels and instructions iteratively
- Support **multiple concurrent annotators** with good task management systems

# Simple interface for labeling text

- The interface depends on your task and data distribution
- Example of interface for a binary labeling task:

```
Please type 1 if this message is disaster-related, or hit Enter if not.  
Type 2 to go back to the last message, type d to see detailed  
definitions, or type s to save your annotations.
```

```
Firefighting continues in Blue Mountains
```

```
> 1
```

## Quick Guidelines for Effective Annotation UI

 **Single focus area** — keep annotators' attention on one part of the screen

 **Hotkeys** — enable fast labeling with keyboard shortcuts

 **Undo/back** — allow error correction without friction

# Managing Machine Learning Data

## Storage Strategies

- **Use databases** (e.g., PostgreSQL, MongoDB) for backups, availability, scalability
- Use local storage for small-scale projects and quick spot-checks
- **Always maintain redundant copies** – data loss is common and costly

## Common Tools

- **DVC** (Data Version Control) – versioning & reproducibility
- **Weights & Biases, MLflow** – dataset tracking and experiment logs
- **Label Studio, Prodigy, Doccano** – UI tools for structured annotation and storage

## Best Practices

- **Never delete labeled data** – archive instead
  - **Be redundant and avoid overwriting.**
- Version data as annotation **guidelines evolve**
- Reproduce and correct older labels if needed for future iterations
- Respect **permissions and privacy**; delete sensitive data when required

# Validation and Evaluation in Active Learning

## Create Validation Data for Hyperparameter Tuning

- Essential to tune models reliably, just like standard ML workflows
- Use **multiple validation sets**, for example:
  - From training data at each iteration
  - From unlabeled data **before** active learning starts
  - From remaining unlabeled data during iterations

## Know Your Data Before Training

- Familiarize yourself with the dataset early on
- **Select evaluation data first** — set it aside before training starts
- Participate in labeling evaluation data personally for better insight

# Always Get Your Evaluation Data First

## What Is Evaluation Data?

- A **random sample** of your dataset (e.g., news headlines)
- Strictly **kept separate** from training data
- Used for **unbiased model accuracy tracking** during active learning

## Why Obtain Evaluation Data Before Sampling?

- Starting sampling methods **before** securing evaluation data causes **bias**

## Pitfalls of delaying evaluation data:

- **Scenario 1 (Sampling by low confidence first):**
  - Evaluation data will be biased towards high-confidence items  $\Rightarrow$  optimistic evaluation
- **Scenario 2 (Pulling from training data after confidence sampling):**
  - Evaluation data will be biased towards low-confidence items  $\Rightarrow$  pessimistic evaluation
- **Scenario 3 (Implementing outlier detection first):**
  - Bias is almost unavoidable, as the selected items have already influenced outlier sampling.

# Always Get Your Evaluation Data First

## What Is Evaluation Data?

- A **random sample** of your dataset (e.g., news headlines)
- Strictly **kept separate** from training data
- Used for **unbiased model accuracy tracking** during active learning

## Why Obtain Evaluation Data Before Sampling?

- Starting sam

## Pitfalls of delaying

- **Scenario 1** (

- Evaluation data will be biased towards high-confidence items  $\Rightarrow$  optimistic evaluation

- **Scenario 2 (Pulling from training data after confidence sampling):**

- Evaluation data will be biased towards low-confidence items  $\Rightarrow$  pessimistic evaluation

- **Scenario 3 (Implementing outlier detection first):**

- Bias is almost unavoidable, as the selected items have already influenced outlier sampling.

***Secure your evaluation dataset upfront to ensure trustworthy and unbiased performance measurement.***

# Model Evaluation in Dynamic Human-in-the-Loop Systems

## The Challenge of Truly Random Data

- **Dynamic streams** (e.g., disaster updates) make random sampling difficult.
- For **finite datasets**, random evaluation sampling ensures unbiased benchmarking.

## Beyond Raw Accuracy

- **Raw accuracy** is misleading with label imbalance (e.g., most headlines are *not* disasters).
- Use **F-score** for the minority class ("disaster-related") as the key metric.

## Confidence Calibration: AUC

- We care not just about predictions, but *how confident* the model is.
- Compute **AUC (Area Under ROC Curve)** to check how well confidence scores reliability

## Transparent Model Naming & Tracking

- Save models with informative filenames:  
`model_2025-05-18_F0.34_AUC0.76_N200.pt`
- Includes: Timestamp, F1-score, AUC, Training size

# Modern Tools for Tracking & Serving

## Many available tools for Machine Learning Operations (MLOps)

| Purpose                                                                                                  | Tool                         | Description                               |
|----------------------------------------------------------------------------------------------------------|------------------------------|-------------------------------------------|
|  Experiment tracking    | Weights & Biases, MLflow     | Log metrics, visualize F-score/AUC trends |
|  Versioning datasets    | DVC, Pachyderm               | Reproduce training/eval splits exactly    |
|  Model serving          | FastAPI, TorchServe, BentoML | Deploy models with confidence scoring     |
|  Performance monitoring | Prometheus + Grafana         | Track accuracy drift post-deployment      |
|  Annotation tools       | Label Studio, Doccano        | Label with timestamps, handle drift       |

# Ensuring Every Data Point Has a Chance: Why Random Sampling Still Matters

## Random Sampling = Baseline + Benchmark

- Include **new random items in each iteration**  
→ Establishes a **performance baseline**
- Enables comparison:
  - Active learning vs. Random selection
  - Tracks model improvement over time

## Measuring Sampling Impact

- How many random samples contradict model predictions?
- Helps **quantify value** of uncertainty/outlier-based strategies

## Fallback for Progress

- When active learning stalls or biases,  
→ **Random samples ensure continued learning**
- Always adds **diverse training signals**

## Avoiding Blind Spots

- Active learning can miss important patterns
- Random sampling ensures **all data has a voice**

## Critical Domains: Disaster Response

- Risk: Ignoring a **life-saving headline** due to biased sampling
- Solution: Maintain **inclusivity & transparency** in selection

# Ensuring Every Data Point Has a Chance: Why Random Sampling Still Matters

## Avoiding Blind Spots

- Active learning can miss important patterns
- Random sampling ensures **all data has a voice**

## Critical Domains: Disaster Response

- Risk: Ignoring a **life-saving headline** due to biased sampling
- Solution: Maintain **inclusivity & transparency** in selection

## Key Takeaways

- Random sampling = scientific control + ethical guardrail
- Helps track, correct, and complement active learning decisions
- Essential in **high-stakes domains (significant real-world consequences)**

# Choosing the Right Sampling Strategy for Imbalanced Data

## Understanding the Challenge

- Disaster-related headlines = **rare**
- Outlier sampling alone → **not effective** for surfacing them
- Strategy needs to **balance discovery and diversity**

## Balanced Sampling Strategy Per Iteration

- 10% randomly selected from unlabeled items
- 80% selected from the lowest confidence items
- 10% selected as outliers

## Expected Outcome Over Time

- With low-confidence items assumed 50:50:  
→ Annotators will label **~40%+ disaster-related** headlines
- Close enough to balance → **Reduces annotator priming** from order effects

# Retrain, Iterate & Improve

## Retraining & Tracking Accuracy

- Add newly annotated items to training set after each iteration
- Retrain model → Observe change in **F-score** and **AUC**
- Tune hyperparameters (e.g., annotations per iteration)
- **Always keep evaluation set fixed** for consistency

## ML + Active Learning Integration

- Many ways to combine model training with sampling logic
- Example: Track which strategy (random / confidence / outlier) selected each sample
- Enables analysis of **bias** and **diversity coverage**

## Post-Iteration Data Analysis (~10 rounds)

- Most samples → from **low confidence**
- **Outlier picks** often discover hidden edge cases / novel examples
- Pay attention to items selected by the outlier strategy – they might highlight missed diversity



## Detect & Correct Sampling Bias

- Active learning self-corrects... but **not always enough**
- Risks:
  - Oversampling very **short/long texts**
  - Bias toward **visual outliers** in image tasks

*Tip: Sample from different “buckets”*

*(e.g., short vs. long, simple vs. complex)*

# Retrain, Iterate & Improve

## Experimenting with Sampling Strategies

- Retrain model on **random samples only**.
- Compare accuracy with models trained on **low-confidence** and **outlier-selected** samples.
- Evaluate each strategy's impact on performance.

## Identifying Next Steps

- Improve annotation interface efficiency.
- Implement quality control for annotations.
- Refine active learning sampling methods.
- Let your experience and data guide priorities.

## Prioritize Data Quality Early

- Focus on **clean, well-annotated data** before complex models.
- Tune ML architectures more in **later iterations**.

# Summary

- A simple system can cover the full cycle: **sampling** → **annotating** → **retraining** for a quick MVP.
- Two easy active learning strategies:
  - Sample **least confident** predictions (uncertainty sampling)
  - Sample **outliers** (diversity sampling)
- Even a **basic command-line interface** can enable efficient human annotation if designed with good interaction principles.
- **Good data management is critical**, especially creating evaluation data upfront to track true model accuracy.
- Regular retraining with new annotations improves model accuracy over iterations.
- Well-designed active learning **self-corrects** over iterations, balancing biases and overfitting.

## ***Reference book***

